

Patient Risk Intelligence MCP Platform

Complete Codebase PRD

Every directory, every file, every contract — a single blueprint to generate the full codebase

Version 1.0 · June 25, 2026

Status: Build-Ready

Table of Contents

1. Executive Summary
2. Problem & Solution
3. Complete Architecture & Workflow
4. Complete Project Directory Structure
5. File-by-File Specification
6. Integration Contracts (Single Source of Truth)
7. Database Schemas
8. Technology Stack & Version Compatibility
9. Build & Run Instructions
10. Acceptance Criteria
11. Appendix — Glossary

1. Executive Summary

This document is a single, complete, role-agnostic blueprint for the Patient Risk Intelligence MCP Platform. It specifies every directory, every file, and every contract needed to generate the codebase end to end — whether by one engineer, a team, or an AI coding agent working from this PRD directly.

The platform lets clinicians ask a natural-language question (“What is this patient's overall risk picture?”) and receive one explainable answer fused from 4 independently governed data domains: vitals, labs/diagnoses, medications/interactions, and clinical notes. It is built entirely from free, self-hosted, open-source components, fed by fully synthetic FHIR patient data (Synthea) — zero real PHI anywhere.

Section 4 is the complete file tree. Section 5 specifies what goes inside each file — responsibilities, function signatures, inputs and outputs — in enough detail to write the file without guessing. Section 6 is the contract every file must honor consistently: token shape, tool names, RBAC matrix, response format.

2. Problem & Solution

2.1 Problem

Clinicians need a unified, real-time, explainable risk picture of a patient. Today the signals are scattered: vitals trends in one system, labs/diagnoses in another, medications in a third, and the richest signals — free-text notes — go unread on every shift. There is no consistent, role-based access control across these systems, and no recorded reason for why PHI was accessed.

2.2 Solution

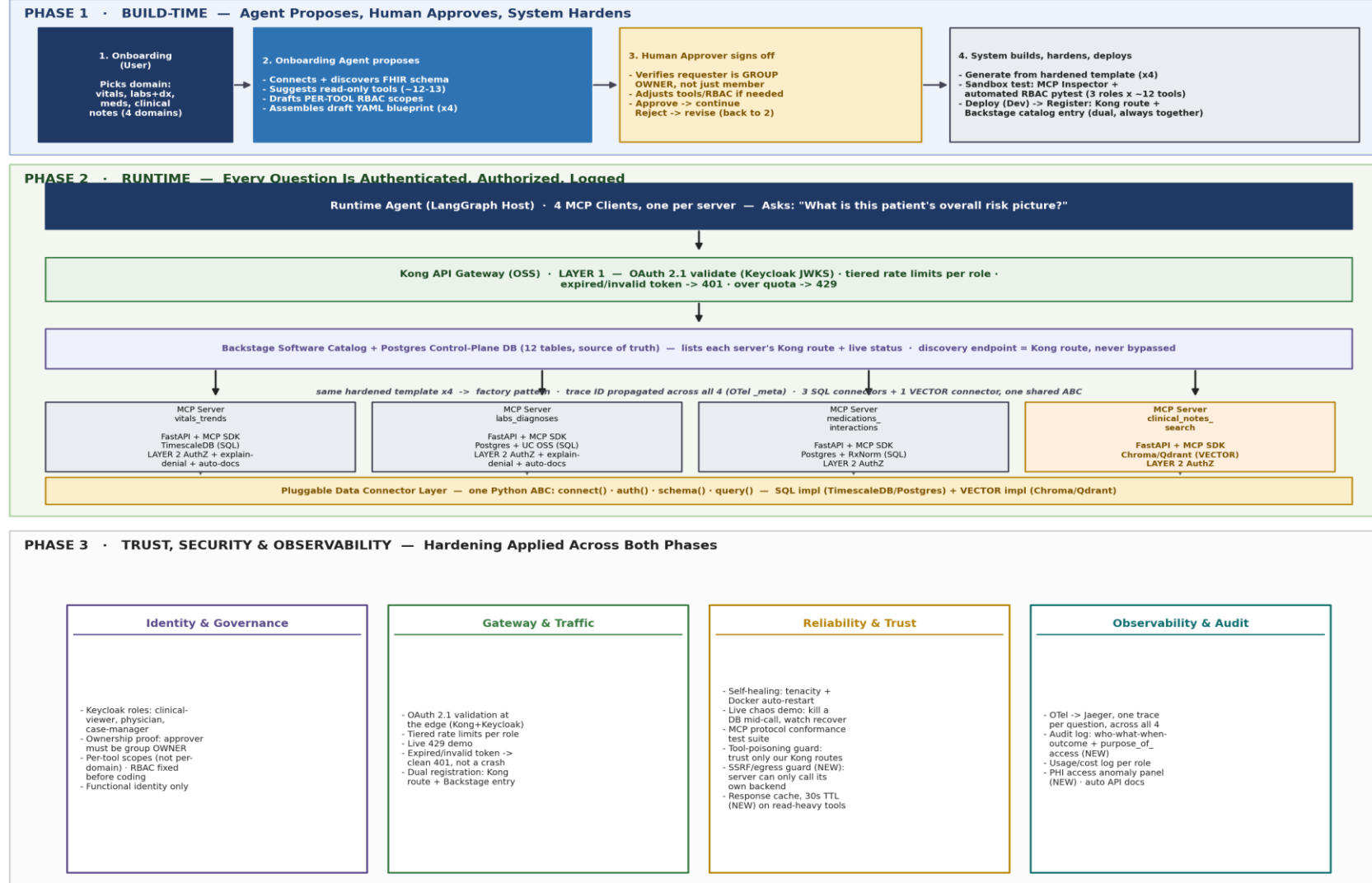
Four independent MCP servers, one per domain, generated from the same hardened template. A runtime agent calls all four and fuses the results into one cited answer. Two connector types — SQL and vector — implement the same interface, proving the architecture is genuinely source-agnostic. A persistent control-plane database is the system's source of truth; a two-layer RBAC model enforces access by role on every single call.

3. Complete Architecture & Workflow

Figure 1 shows the full system in three phases: Build-Time (a new domain is proposed, approved, and hardened into a server), Runtime (a live question is answered across all 4 servers), and Trust/Observability (the controls applied throughout). Every box in this diagram corresponds to a directory or file in Section 4 — nothing in the diagram is undocumented in the file tree, and nothing in the file tree exists outside this diagram.

Patient Risk Intelligence MCP Platform — End-to-End Architecture & Workflow

Healthcare Clinical Risk Intelligence POC · MCP spec 2025-11-25, Streamable HTTP transport · FastAPI + MCP SDK v1.28.x



All tool outputs shaped as FHIR R4 resources (Observation, MedicationStatement, Condition, DocumentReference) · LOINC/RxNorm/SNOMED-CT codes carried through · NEWS2 deterioration scoring · drug-interaction checks use a curated open rule set, not a licensed clinical DB · synthetic patients via Synthea (zero real PHI).

Figure 1 — Complete target architecture. Every component shown has a corresponding directory/file in Section 4.

3.1 Phase 1 — Build-Time

- Onboarding (User) picks a domain → Onboarding Agent (backend/onboarding_agent/) discovers schema, suggests tools, drafts RBAC, writes a YAML blueprint.
- Human Approver reviews the blueprint file directly — no UI required for this PRD's scope — and signs off or requests revision.
- The hardened template (backend/servers/*/main.py, sharing backend/shared/*) is instantiated per domain, tested via MCP Inspector + the RBAC pytest suite, then registered (Kong route + registry-db entry).

3.2 Phase 2 — Runtime

- Runtime Agent (agent/runtime_agent.py) receives a question, attaches the caller's token, opens 4 MCP Clients.
- Kong (infra/kong/kong.yml) validates the token, applies tiered rate limits, routes to the right server.
- Each MCP server (backend/servers/<domain>/) re-verifies the token (backend/shared/auth.py), checks per-tool scope, queries its connector (backend/connectors/), and returns a FHIR-shaped result or an explained denial.

3.3 Phase 3 — Trust & Observability

- Every call passes through backend/shared/egress_guard.py (SSRF guard), backend/shared/cache.py (30s TTL), backend/shared/audit.py (structured log with purpose_of_access), and backend/shared/telemetry.py (OpenTelemetry trace propagation).
- Self-healing (Docker restart policies + tenacity retries) and the registry-db (12 tables) provide the operational source of truth surfaced in the frontend dashboard.

4. Complete Project Directory Structure

One repository. Every file below is specified in Section 5 — nothing in this tree is unaccounted for, and nothing in Section 5 refers to a file not listed here.

```

patient-risk-intelligence/
├── README.md
├── docker-compose.yml          # all 16 services
├── .env.example
├── infra/
│   ├── kong/
│   │   └── kong.yml           # routes, JWT, rate limits
│   ├── keycloak/
│   │   └── realm-export.json  # 3 roles, OAuth client
│   ├── unitycatalog/
│   │   └── Dockerfile         # wraps UC OSS server jar
│   ├── synthea/
│   │   ├── Dockerfile
│   │   ├── load_patients.py   # generate + load + embed
│   │   └── demo_patient_aliases.json # NEW – friendly ID map
│   └── postgres/
│       ├── init-multiple-dbs.sh
│       ├── init-timescale-vitals.sql
│       ├── init-labs-diagnoses.sql
│       ├── init-medications.sql
│       └── init-registry-db.sql # 12-table schema
├── backend/
│   ├── shared/
│   │   ├── connector_base.py  # used by all 4 servers
│   │   ├── connector_base.py  # Connector ABC
│   │   ├── fhir_shape.py      # tool output -> FHIR R4
│   │   ├── auth.py            # RBAC engine (Layer 2)
│   │   ├── audit.py           # audit + purpose_of_access
│   │   ├── telemetry.py       # OTel + trace propagation
│   │   ├── tool_trust.py      # tool-poisoning guard
│   │   ├── usage_log.py       # usage/cost log per role
│   │   ├── egress_guard.py     # SSRF/egress guard
│   │   └── cache.py           # 30s TTL response cache
│   ├── connectors/
│   │   ├── sql_connector.py   # TimescaleDB/Postgres
│   │   └── vector_connector.py # Qdrant
│   └── servers/
│       ├── vitals_trends/
│       │   ├── main.py        # FastAPI + MCP SDK
│       │   ├── tools.py       # vitals tools
│       │   ├── news2.py       # NEWS2 scoring
│       │   ├── blueprint.yaml
│       │   ├── Dockerfile
│       │   └── requirements.txt
│       └── labs_diagnoses/
│           ├── main.py
│           ├── tools.py       # labs/diagnosis tools
│           ├── blueprint.yaml
│           ├── Dockerfile
│           └── requirements.txt

```

```

├── medications_interactions/
│   ├── main.py
│   ├── tools.py          # medication tools
│   ├── interactions.py   # curated RxNorm rules
│   ├── blueprint.yaml
│   ├── Dockerfile
│   └── requirements.txt
├── clinical_notes_search/
│   ├── main.py          # uses vector_connector
│   ├── tools.py         # search + get_notes_by_type
│   ├── blueprint.yaml
│   ├── Dockerfile
│   └── requirements.txt
├── onboarding_agent/    # build-time, ONE process
│   ├── discover.py      # schema/FHIR discovery
│   ├── suggest_tools.py # LLM: schema -> tools
│   ├── draft_rbac.py    # drafts per-tool scopes
│   └── assemble_blueprint.py # writes YAML only
├── registry/
│   ├── main.py          # registry-api app
│   ├── auth.py          # NEW – non-anonymous check
│   ├── models.py        # ORM for 12 tables
│   ├── Dockerfile
│   └── requirements.txt
├── tests/
│   ├── test_rbac_matrix.py    # 3 roles x ~12 tools
│   ├── test_self_healing.py  # chaos demo
│   └── test_mcp_conformance.py
├── agent/                 # Runtime Agent (MCP Host)
│   ├── runtime_agent.py    # 4 MCP Clients, fusion
│   ├── prompts.py          # citation-based fusion
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── package.json
│   ├── tailwind.config.ts
│   ├── Dockerfile
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── chat/page.tsx    # CopilotKit chat
│   │   ├── dashboard/page.tsx # Tremor dashboard
│   │   └── api/auth/
│   │       └── [...nextauth]/route.ts # NEW – login
│   ├── components/
│   │   ├── ApprovalCard.tsx # human-in-the-loop
│   │   └── RegistryTable.tsx # live registry status
├── .github/workflows/
│   └── ci.yml              # build -> test -> deploy
├── docs/
│   ├── patient_risk_architecture_workflow.png
│   └── prd.docx

```

5. File-by-File Specification

Specifications below give each file's responsibility and the function/class signatures it must expose. Where a file's content is config rather than code, the actual file is provided in full (Section 7 for SQL; the docker-compose.yml and kong.yml referenced here are delivered as complete, runnable files alongside this PRD).

5.1 Infrastructure & Config

infra/kong/kong.yml

Declarative Kong config (format 3.0). 4 services, one per MCP server, each with a route under /mcp/clinical/{domain}/dev, a jwt plugin (validates against Keycloak's JWKS), and a rate-limiting plugin. medications_interactions and clinical_notes_search get a tighter quota (30/min) than vitals_trends and labs_diagnoses (60/min) — this is the 'tiered rate limits per role' requirement in practice, expressed at the route level.

infra/keycloak/realm-export.json

- Realm: patient-risk
- 3 client roles: grp-clinical-viewer, grp-physician, grp-case-manager
- OAuth 2.1 client configured for the Runtime Agent (client_credentials or password grant, POC-appropriate)
- Token mapper ensuring groups[] and scp claims are included in the issued JWT, matching Section 6.1 exactly

infra/postgres/*.sql

Full schemas given in Section 7 — init-timescale-vitals.sql, init-labs-diagnoses.sql, init-medications.sql, init-registry-db.sql, plus init-multiple-dbs.sh (standard community script creating one database per name in POSTGRES_MULTIPLE_DATABASES).

infra/synthea/load_patients.py

- run_synthea(patient_count: int, seed: int) -> Path — invokes the Synthea jar with a FIXED seed (gap fix — previously unseeded, so the demo patient referenced in docs/scripts wasn't guaranteed to exist on a reseed), returns the output directory of generated FHIR bundles
- load_vitals(bundle, conn) — extracts Observation(vital-signs) resources, inserts into vitals
- load_labs_and_diagnoses(bundle, conn) — extracts Observation(labs) and Condition resources
- load_medications(bundle, conn) — extracts MedicationStatement resources
- embed_and_load_notes(bundle, qdrant_client) — extracts DocumentReference/narrative text, tags note_type='physician_note' and author_role='physician' by default (Synthea writes notes from the encounter provider's perspective), embeds with sentence-transformers/all-MiniLM-L6-v2, upserts into Qdrant with patient_id/note_date/author/note_type/author_role metadata
- write_demo_aliases(patients: list) -> Path — NEW (gap fix): writes demo_patient_aliases.json mapping friendly IDs (e.g. demo-patient-1) to the real generated patient UUIDs, so a fixed seed + this file together make the demo reproducible
- main() — orchestrates all of the above; this is the container's entrypoint, runs once, exits

5.2 Shared Backend Modules (backend/shared/)

Every one of the 4 MCP servers imports from this package. These modules ARE the Fixed Core referenced in the architecture — changing them changes all 4 servers at once.

connector_base.py

```
class Connector(ABC):
    async def connect(self) -> None: ...
    async def auth(self) -> None: ...
    async def schema(self) -> dict: ...
    async def query(self, params: dict) -> list[dict]: ...
```

fhir_shape.py

- `to_observation(row: dict, loinc_code: str) -> dict` — wraps a vitals/labs row as a FHIR Observation resource
- `to_medication_statement(row: dict) -> dict` — wraps a medication row as MedicationStatement
- `to_condition(row: dict) -> dict` — wraps a diagnosis row as Condition
- `to_document_reference(note: dict) -> dict` — wraps a clinical note as DocumentReference

auth.py — Layer 2 AuthZ engine

- `verify_token(token: str, jwks_url: str) -> dict` — re-verifies signature/audience/expiry, returns decoded claims
- `check_scope(claims: dict, required_scope: str) -> bool` — intersects `claims['groups']` / `claims['scp']` against the tool's required scope
- `authorize_or_raise(claims: dict, required_scope: str) -> None` — raises an `HTTPException(403)` with `{"error": {"code": "forbidden", "reason": f"missing scope {required_scope}"}}` on failure — this is the explain-denial behavior

audit.py

- `PURPOSE_OF_ACCESS = {deterioration_review, medication_reconciliation, discharge_planning, care_coordination, routine_review}` — a fixed enum, not free text (gap fix); `routine_review` is the default
- `log_call(who, what, outcome, reason, purpose_of_access, trace_id) -> None` — raises `ValueError` if `purpose_of_access` isn't in `PURPOSE_OF_ACCESS`; writes one row matching the schema in Section 6.4, to stdout and to registry-db's `audit_events` table

telemetry.py

- `init_tracer(service_name: str) -> None` — OpenTelemetry setup, exports to Jaeger
- `trace_id_from_request(request) -> str` — extracts/continues the trace ID the Runtime Agent originates, so one question's full path is one trace across all 4 servers

tool_trust.py

- `verify_kong_origin(request) -> bool` — confirms the request arrived via this server's registered Kong route, not a direct call bypassing the gateway

usage_log.py

- `record_usage(role: str, tool: str, outcome: str) -> None` — increments a per-role, per-week counter (query count and denial count) used by the dashboard

`egress_guard.py`

- `locked_connector_for(server_name: str) -> Connector` — returns a Connector instance pre-bound to this server's configured backend at startup; the server has no code path that can construct a Connector pointed at an arbitrary URL

`cache.py`

- `cached(ttl_seconds: int = 30)` — decorator wrapping a tool function; caches by (tool_name, params) for the TTL window

5.3 Connector Layer (backend/connectors/)

sql_connector.py

- class SQLConnector(Connector) — used by vitals_trends (TimescaleDB), labs_diagnoses, medications_interactions (Postgres)
- connect() opens an asyncpg/SQLAlchemy pool to the URL in DB_URL
- schema() introspects information_schema for the configured tables
- query(params) runs a parameterized, read-only SELECT — never raw string interpolation, to prevent injection and to make the query guardrails (Section 6.5) enforceable

vector_connector.py

- class VectorConnector(Connector) — used by clinical_notes_search (Qdrant)
- connect() opens a Qdrant client against QDRANT_URL
- schema() returns the collection's vector size + metadata field names
- query(params) embeds params['query_text'] with sentence-transformers/all-MiniLM-L6-v2 — the EXACT same model/version as the loader — then runs a similarity search filtered by params.get('patient_id') and optionally params.get('note_type')

5.4 MCP Servers (backend/servers/)

All 4 share one pattern: a FastAPI app wrapping the MCP Python SDK, Streamable HTTP transport, Layer-2 auth on every tool call, FHIR-shaped responses.

vitals_trends/main.py + tools.py + news2.py

Tool	Signature	Behavior
get_vitals_trend	(patient_id: str, hours: int = 24) -> list[Observation]	Queries vitals via SQLConnector, FHIR-shapes each row, cached 30s
compute_news2_score	(patient_id: str) -> dict	Pulls latest vitals, applies the published NEWS2 algorithm (news2.py), returns score + risk band
list_abnormal_vitals	(patient_id: str, hours: int = 24) -> list[Observation]	Same query, filtered to values outside normal range

news2.py: compute_news2(resp_rate, spo2, temp, systolic_bp, heart_rate, consciousness) -> int — a direct, unmodified implementation of the published NHS NEWS2 scoring table.

labs_diagnoses/main.py + tools.py

Tool	Signature	Behavior
get_lab_trend	(patient_id: str, test_name: str None) -> list[Observation]	Queries labs, optionally filtered by test name
get_active_diagnoses	(patient_id: str) -> list[Condition]	Queries diagnoses where onset_date is set and no resolution recorded
get_diagnosis_history	(patient_id: str) -> list[Condition]	All diagnoses, ordered by onset_date

medications_interactions/main.py + tools.py + interactions.py

Tool	Signature	Behavior
get_active_medications	(patient_id: str) -> list[MedicationStatement]	Queries medications where end_date is null or future
check_drug_interactions	(patient_id: str) -> list[dict]	Cross-references active medications' rxnorm_code pairs against interaction_rules
get_polypharmacy_risk	(patient_id: str) -> dict	Simple count-based flag: 5+ active medications = elevated risk

interactions.py: check_pairs(rxnorm_codes: list[str]) -> list[dict] — looks up interaction_rules; explicitly documented as a curated illustrative rule set, not a licensed clinical drug-interaction database.

clinical_notes_search/main.py + tools.py

Tool	Signature	Behavior
semantic_search_notes	(patient_id: str, query_text: str, top_k: int)	Embeds query_text, similarity search via VectorConnector, filtered to patient_id

	int = 5) -> list[DocumentReference]	
get_recent_notes	(patient_id: str, limit: int = 5) -> list[DocumentReference]	Most recent notes by note_date, no semantic ranking
get_notes_by_type	(patient_id: str, note_type: str) -> list[DocumentReference]	Direct filter by note_type (physician_note/nursing_note/discharge_summary/consult_note) — not semantic

Gap fix: every note's Qdrant payload now carries note_type and author_role (physician/nurse/case_manager) — previously these were conflated (a note could be tagged with an author but no type, making 'show me doctor notes' impossible to answer precisely). Synthea generates one note per encounter from the provider's perspective; the loader tags these physician_note/physician by default. Any nursing_note/discharge_summary variants are synthetic augmentation, explicitly marked as such, not Synthea-native.

Critical consistency rule: vector_connector.py's query() must embed search text with the exact same model and version used by infra/synthea/load_patients.py at load time (sentence-transformers/all-MiniLM-L6-v2). A mismatch produces no error — just silently meaningless similarity results.

This server's main.py is the one that imports VectorConnector instead of SQLConnector — everything else (auth, audit, egress guard, cache, FHIR shaping) is identical to the other 3 servers.

5.5 Onboarding Agent (backend/onboarding_agent/)

One process, four internal stages, run once per new domain. It never deploys anything — its only output is a YAML file.

- `discover.py`: `discover_schema(connector: Connector) -> dict` — introspects the target source, returns table/column or collection/field metadata
- `suggest_tools.py`: `suggest_tools(schema: dict) -> list[dict]` — one LLM call turning schema into draft tool name + description pairs
- `draft_rbac.py`: `draft_rbac(tools: list[dict], role_matrix: dict) -> dict` — applies the fixed role matrix (Section 6.3) to produce per-tool scopes
- `assemble_blueprint.py`: `assemble_blueprint(domain, schema, tools, rbac) -> Path` — writes `blueprint.yaml`; this is the human approval artifact

5.6 Registry API (backend/registry/)

- `main.py`: FastAPI app exposing GET `/servers`, GET `/servers/{id}/health`, GET `/audit?role=`, POST `/servers` (called by the build pipeline on registration)
- `auth.py`: NEW (gap fix) — `require_valid_token(request) -> dict`, rejects any request without a valid, correctly signed, non-expired Keycloak token. Previously this API had no authentication at all.
- `models.py`: SQLAlchemy ORM models mirroring the 12 tables in `init-registry-db.sql` exactly — see Section 7.4

5.7 Runtime Agent (agent/)

The MCP Host. Holds 4 MCP Clients, one per server, opened against the routes in Section 6.2.

- `runtime_agent.py`: class `RuntimeAgent` — `__init__` opens all 4 clients; `ask(question: str, token: str) -> str` orchestrates tool selection, calls relevant tools in parallel, passes results + question to the LLM for fused, cited synthesis
- `prompts.py`: `SYNTHESIS_PROMPT` — instructs the LLM to cite which server each fact came from (e.g. '(vitals_trends)') and to never state a fact without a citation
- Exposes its own small HTTP API (POST /ask) on port 8500 so the frontend can call it

5.8 Frontend (frontend/)

- `app/api/auth/[...nextauth]/route.ts`: NEW (gap fix) — NextAuth.js configured with Keycloak as an OIDC provider. Previously there was no documented path for a user's identity to reach the Runtime Agent at all.
- `app/chat/page.tsx` — CopilotKit chat UI calling the Runtime Agent's `/ask` endpoint with the NextAuth session's access token as a Bearer header; includes a reason-for-query selector offering the 5 `purpose_of_access` values (Section 6.3); renders `ApprovalCard.tsx` when an onboarding approval is pending (`useHumanInTheLoop`)
- `app/dashboard/page.tsx` — Tremor dashboard: KPI cards (questions asked, denials) from `usage_log` data via the registry API; `RegistryTable.tsx` showing live server/route status from `GET /servers`
- `components/ApprovalCard.tsx` — renders a draft blueprint's tools/RBAC with Approve/Reject buttons
- `components/RegistryTable.tsx` — polls `GET /servers` every few seconds, renders status (healthy/unhealthy) per server

5.9 Tests (backend/tests/)

- `test_rbac_matrix.py` — parametrized over (role, tool) for all 3 roles x ~12 tools; asserts the response code matches the matrix in Section 6.3 exactly
- `test_self_healing.py` — programmatically stops a database container mid-call, asserts the connector retries and the server's health endpoint recovers within a bounded time
- `test_mcp_conformance.py` — runs each server against the official MCP conformance suite

5.10 CI/CD (.github/workflows/ci.yml)

Stages: validate → generate spec (if onboarding changed) → security scan → build images → unit tests → sandbox integration tests (docker compose up against a test profile) → deploy.

6. Integration Contracts (Single Source of Truth)

Every file in Section 5 that touches auth, tools, or RBAC must match this section exactly. If anything here changes, every affected file in Section 5 changes with it.

6.1 Token Shape

```
{ "sub": "<user id>", "oid": "<object id, used in audit 'who'>",
  "groups": ["grp-clinical-viewer" | "grp-physician" | "grp-case-manager"],
  "scp": "mcp.vitals.read mcp.labs.read ..." } (space-separated scopes)
```

6.2 Tool Names, Scopes, and Kong Routes

Server	Tools	Scope	Kong route
vitals_trends	get_vitals_trend, compute_news2_score, list_abnormal_vitals	mcp.vitals.read	/mcp/clinical/vitals-trends/dev
labs_diagnoses	get_lab_trend, get_active_diagnoses, get_diagnosis_history	mcp.labs.read	/mcp/clinical/labs-diagnoses/dev
medications_interactions	get_active_medications, check_drug_interactions, get_polypharmacy_risk	mcp.meds.read	/mcp/clinical/medications-interactions/dev
clinical_notes_search	semantic_search_notes, get_recent_notes, get_notes_by_type	mcp.notes.read	/mcp/clinical/clinical-notes-search/dev

6.2.1 Note Typing (gap fix)

Every `clinical_notes_search` payload carries `note_type` (`physician_note` | `nursing_note` | `discharge_summary` | `consult_note`) and `author_role` (`physician` | `nurse` | `case_manager`) — these were previously conflated, making it impossible to precisely answer “show me doctor notes.” Synthea-native notes default to `physician_note/physician`; any other type is synthetic augmentation, explicitly marked as such.

6.3 RBAC Matrix

Role	vitals_trends	labs_diagnoses	medications_interactions	clinical_notes_search
clinical-viewer	Allow	Allow	Deny	Deny
physician	Allow	Allow	Allow	Allow
case-manager	Deny	Deny	Deny	Allow

6.4 Audit Log Schema

```
{
  "who": "<oid from token>", "what": "<tool name called>",
```

```
"when": "<ISO 8601 timestamp>", "outcome": "200 | 401 | 403 | 429",
"reason": "<populated on 403>", "purpose_of_access": "<see enum below>",
"trace_id": "<OpenTelemetry trace id>" }
```

purpose_of_access is a fixed enum, not free text (gap fix): deterioration_review, medication_reconciliation, discharge_planning, care_coordination, routine_review (default). audit.py rejects any other value.

6.4.1 Frontend Authentication (gap fix)

NextAuth.js (frontend) authenticates the user against Keycloak via OIDC. The resulting access token — same shape as Section 6.1 — is attached as a Bearer header on every call to the Runtime Agent's /ask endpoint, which forwards it to each MCP server through Kong. registry-api requires this same token to be present and valid (non-anonymous) on every request; it does not need to be backed by a specific role, just not absent.

6.5 Response Envelope

Success: a FHIR R4 resource (Observation / MedicationStatement / Condition / DocumentReference). Denial: HTTP 403 with { "error": { "code": "forbidden", "reason": "missing scope <scope>" } }.

6.6 Policy Guardrails

- Query: block DROP/DELETE/UPDATE/INSERT/MERGE unless explicitly approved; limit result rows; limit execution time; mask PHI columns (patient name, MRN, DOB).
- Vector: limit sample size per semantic search; block raw document download unless approved; log every read.
- Agent: only approved agents call production servers; human-in-the-loop for anything touching identifiable PHI; per-agent rate limits.

7. Database Schemas

Full, runnable SQL — delivered as actual files alongside this PRD (infra/postgres/*.sql) and reproduced here for completeness.

7.1 vitals (TimescaleDB)

```
CREATE EXTENSION IF NOT EXISTS timescaledb;
CREATE TABLE vitals (
  patient_id TEXT NOT NULL, recorded_at TIMESTAMPTZ NOT NULL,
  heart_rate INT, systolic_bp INT, diastolic_bp INT, spo2 INT,
  resp_rate INT, temperature_c NUMERIC(4,1), loinc_code TEXT,
  PRIMARY KEY (patient_id, recorded_at));
SELECT create_hypertable('vitals', 'recorded_at');
```

7.2 labs_diagnoses (Postgres)

```
CREATE TABLE labs (lab_id SERIAL PRIMARY KEY, patient_id TEXT NOT NULL,
  test_name TEXT NOT NULL, loinc_code TEXT, result_value NUMERIC,
  unit TEXT, reference_range TEXT, abnormal_flag TEXT,
  collected_at TIMESTAMPTZ NOT NULL);
CREATE TABLE diagnoses (diagnosis_id SERIAL PRIMARY KEY,
  patient_id TEXT NOT NULL, icd10_code TEXT, snomed_code TEXT,
  description TEXT, diagnosis_type TEXT, onset_date DATE);
```

7.3 medications_interactions (Postgres)

```
CREATE TABLE medications (med_id SERIAL PRIMARY KEY, patient_id TEXT NOT NULL,
  drug_name TEXT NOT NULL, rxnorm_code TEXT, dose TEXT, route TEXT,
  frequency TEXT, start_date DATE, end_date DATE, prescriber TEXT);
CREATE TABLE interaction_rules (rule_id SERIAL PRIMARY KEY,
  rxnorm_code_a TEXT NOT NULL, rxnorm_code_b TEXT NOT NULL,
  severity TEXT, description TEXT);
```

7.4 registry (Postgres) — 12-table control plane

products, connectors, mcp_servers, tool_specs, rbac_mappings, deployment_environments, gateway_routes, api_registry, agent_integrations, audit_events, health_checks, schema_snapshots — full CREATE TABLE statements in infra/postgres/init-registry-db.sql (delivered in full alongside this PRD; foreign keys chain products → connectors → mcp_servers → tool_specs/rbac_mappings).

8. Technology Stack & Version Compatibility

Component	Choice	Version / Note
MCP protocol spec	Model Context Protocol	2025-11-25 (current stable)
MCP SDK (Python)	mcp	v1.28.x — pin mcp>=1.27,<2 across ALL 4 servers + agent
Transport	Streamable HTTP	—
Language runtime	Python	3.12 or 3.13
Web framework	FastAPI	0.136.x — requires Pydantic v2
Time-series DB	TimescaleDB	Postgres 16 extension
Relational DB	PostgreSQL	16
Vector DB	Qdrant	built-in dashboard at :6333/dashboard
Embedding model	sentence-transformers/all-MiniLM-L6-v2	MUST match exactly between loader and query — see Section 6.2.1
Data governance (optional)	Unity Catalog OSS	Apache 2.0, LF AI & Data
Synthetic data	Synthea	MITRE, generates FHIR R4 — run with a fixed SYNTHEA_SEED
Clinical scoring	NEWS2	published NHS algorithm
Medication terminology	RxNorm	NLM, free/public
API Gateway	Kong Gateway OSS	3.14
Identity Provider	Keycloak	26.0+
Frontend auth	NextAuth.js	Keycloak as OIDC provider — see Section 6.4.1
Agent orchestration	LangGraph	—
Frontend	Next.js + React + Tailwind + CopilotKit + Tremor	—
Tracing / Metrics	OpenTelemetry → Jaeger / Prometheus + Grafana	—
Resilience	tenacity + Docker restart policies	—
CI/CD	GitHub Actions	—
Orchestration	Docker Compose	16 services, single host

Hard rule: every MCP server and the Runtime Agent must pin the exact same mcp SDK version. Mixing versions causes protocol-negotiation failures between the agent's Clients and the servers.

9. Build & Run Instructions

- 1. `cp .env.example .env` and fill in real passwords
- 2. `docker compose build` — builds all custom images (4 servers, agent, frontend, synthea-loader, registry-api, unitycatalog)
- 3. `docker compose up -d` — starts all persistent services
- 4. `docker compose run synthea-loader` — one-time (or re-run anytime) data seed
- 5. Open <http://localhost:5050> (pgAdmin) and <http://localhost:6333/dashboard> (Qdrant) to verify data landed
- 6. Open <http://localhost:3000> (frontend) to use the chat and dashboard
- 7. `pytest backend/tests/` — run the RBAC matrix, self-healing, and conformance tests

10. Acceptance Criteria

- A physician token successfully calls all 4 servers and receives one cited, synthesized answer.
- A clinical-viewer token is correctly denied on `medications_interactions` and `clinical_notes_search`, with an explicit missing-scope reason.
- `compute_news2_score` matches the published NEWS2 algorithm for a known synthetic vitals set.
- A repeated call beyond a role's Kong quota returns 429; an expired token returns a clean 401.
- Killing a database container mid-call triggers visible retry and automatic container restart.
- Every call — allowed or denied — produces a complete audit record including `purpose_of_access`.
- The RBAC pytest suite passes all 12 role/tool combinations.
- `registry-db` reflects the live, current state of all 4 registered servers.
- `get_notes_by_type` correctly distinguishes `physician_note` from other note types.
- A semantic search returns relevant results, confirming the loader and query embedding models match.
- Re-running the Synthea loader with the same `SYNTHEA_SEED` reproduces the same demo patient via `demo_patient_aliases.json`.
- An audit record with an invalid `purpose_of_access` value is rejected, not silently logged.
- A user logs in via NextAuth/Keycloak and that session's token successfully authenticates their first question.
- `registry-api` rejects a request with no token; accepts one with a valid token.

11. Appendix — Glossary

Term	Definition
Host	The application the agent runs inside — here, the Runtime Agent process.
Client	Lives inside the Host; manages the connection to exactly one Server. 4 in this platform.
Server	Exposes Tools over MCP. 4 independent instances, one per data domain.
FHIR R4	HL7 standard for healthcare data exchange — the shape every tool response uses.
LOINC / RxNorm / SNOMED-CT	Standard terminology codes for labs/vitals, medications, and diagnoses respectively.
NEWS2	Published NHS early-warning deterioration score.
Synthea	MITRE's open-source synthetic FHIR patient generator — the platform's only data source.